

第 01 章 Shell 解析、引用、展开与命令组合

从输入字符到最终 argv：把引用、变量、glob、环境与退出状态放进同一条推理链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：Shell 不会把输入行原样交给程序。它先识别词和操作符，再按引用上下文完成展开、拆分和路径名匹配，最终形成命令名、参数列表、执行环境与退出状态。

- 01 识别词与引用判断哪些字符仍是 Shell 语法
- 02 参数与命令替换把变量值或命令输出嵌入当前词
- 03 词拆分未引用展开结果可能变成多个参数
- 04 路径名展开glob 模式可能变成多个真实路径
- 05 最终 `argv` 确认参数数量、顺序与精确文本
- 06 执行与状态区分环境边界并保存退出状态

专题地图

- 1. 知识专题从输入文本到最终参数列表
- 2. 知识专题引用与转义：决定哪些字符还是语法
- 3. 知识专题展开流水线：一个词为什么改变内容或数量
- 4. 操作专题Shell 变量、导出属性与子进程环境
- 5. 操作专题从目标参数反向构造安全命令
- 6. 知识专题退出状态与命令组合
- 7. 知识专题当前 Shell 与子 Shell
- 8. 诊断专题从异常结果反推解析阶段

阅读时持续回答

- 1. 哪些字符仍属于 Shell 语法？
- 2. 哪些内容会发生展开？
- 3. 展开结果会不会继续被拆分？
- 4. glob 会生成几个真实路径？
- 5. 程序最终获得几个参数？
- 6. 变量是否进入子进程环境？
- 7. 哪一条命令的退出状态被保存？
- 8. 状态修改发生在当前 Shell 还是子 Shell？

章节边界

重定向、文件描述符与管道留给第 02 章；本地帮助体系完整展开留给第 03 章；本章不扩展为完整 Bash 脚本编程课程。

输入行不是程序参数：先重建 Bash 的处理链

你在终端里看到的是一行字符，程序真正接收到的却是一组参数。Bash 会先识别词和控制操作符，再根据引用上下文执行参数展开、命令替换、词拆分和路径名展开，最后才选择命令并执行。很多看似“命令不会用”的故障，本质上是程序从一开始就收到了错误数量或错误内容的参数。

```
name='report final'
printf '<%s>\n' $name
printf '<%s>\n' "$name"
printf '<%s>\n' '$name'
```

这三条命令分别可能产生两个参数、一个展开后的参数和一个字面的 `$name`。本章不把“变量外面最好加引号”当作口号，而是要求你始终能回答：输入中的哪些字符仍是语法、哪些内容会被替换、替换结果是否还会再拆分、模式会匹配多少路径、程序最终获得多少参数，以及命令结束后哪一个状态被保留。

源文本 → Shell 词 → 引用上下文 → 参数展开 / 命令替换 → 词拆分 → glob → 最终 argv → 执行环境 → 退出状态

核心概念：先把对象和边界分清

概念

Shell 词与最终 argv Shell 词是 Bash 解析命令行时识别的语法单位，它可能最终成为命令名或一个参数；最终 argv 则是展开、拆分、路径名匹配和引用移除之后真正交给程序的字符串数组。源文本里看见三个“片段”不代表程序一定收到三个参数，反过来也一样。最可靠的证据不是目测命令行，而是用参数观察器确认参数数量和每个参数的精确文本。

概念

引用上下文 单引号、双引号与反斜杠不是执行前最后添加的保护壳，而是从解析阶段开始改变字符是否仍具有语法含义。单引号使内部字符保持字面值；双引号允许参数展开和命令替换，但通常保持展开结果为一个词；反斜杠只保护紧随其后的一个字符。引用只能保证 Shell 层的参数边界，不能证明路径存在、权限正确或业务目标完成。

概念

参数展开 参数展开用 `$name` 或 `${name}` 读取当前 Shell 中的变量值。花括号最重要的基础作用是明确变量名边界，例如 `${name}01` 表示变量 `name` 后接字面 `01`。未设置、已设置为空、未引用空展开和显式空参数并不是同一状态，必须通过最终参数数量来区分。

概念

命令替换 `$(command)` 会在子 Shell 环境中执行命令，并把其标准输出嵌入当前词；末尾连续换行会被移除。命令替换捕获的是文本，不是“成功或失败”这一状态。若替换结果应当保持一个参数，应把整个替换放在双引号中；若希望修改当前 Shell 的目录或变量，则不能依赖命令替换内部的状态变化。

概念

词拆分 词拆分主要作用于未引用的参数展开和命令替换结果，并依据 `IFS` 将文本分成字段。因此变量值中已有空格时，`$value` 可能生成多个参数，而 `"$value"` 通常保持一个参数。源文本中的空格属于较早的词法边界，变量值中的空格是否再次拆分则由引用上下文决定。

概念

路径名展开 (glob) `glob` 由 Shell 在目标程序启动前完成，`*`、`?` 和字符集合模式可能被替换成零个、一个或多个路径。模式被双引号包住后不会展开；默认情况下 `*` 通常不匹配名称开头的点，无匹配模式通常会保留为字面文本。Shell 选项和 `locale` 会改变部分边界，因此异常时要先调查当前 Shell 状态。

概念

Shell 变量与导出属性 Shell 变量属于当前 Shell 的状态；`export` 添加的是“随后启动的子进程可以继承”这一属性。`set`、`export -p` 和 `env` 观察的对象不同：当前 Shell 状态、带导出属性的变量，以及当前进程环境。子进程获得的是启动时的环境副本，不能反向修改父 Shell。

概念

退出状态 命令或命令列表完成后会向 Shell 返回一个整数状态，`0` 通常表示按该命令自身约定成功，非零表示未成功。`$?` 只保存最近一条命令或命令列表的状态，会被下一条命令立即覆盖；`&&` 与 `||` 正是依据这个状态短路执行。命令状态为零只能证明局部接口成功，不能自动证明完整业务终态正确。

概念

当前 Shell 与子 Shell { list; } 在当前 Shell 中执行，变量和工作目录修改可能保留；(list) 与 \$(...) 建立子 Shell 边界，内部状态通常不会回写父 Shell。选择分组形式的核心不是括号外观，而是任务是否要求保留状态，以及是否只需要捕获文本或隔离影响。

操作语义速查：先记住关键入口的接口形状

这里不是完整 man page，而是本章最关键的查询、构造和验证入口。进入正文后，每个专题再按对象、证据和边界展开。

type / type -a

SYNOPSIS

```
<pre><code>type [-afptP] name [name ...]</code></pre>
<p>确认当前 Bash 如何解析一个名称，区分 alias、函数、builtin 和 <code>PATH</code> 中的外部命令。
</p>
<dl><dt><code>type name</code></dt><dd>显示当前优先解析结果。</dd><dt><code>-a</code></dt>
<dd>尽可能列出所有可用入口。</dd><dt><code>-P</code></dt><dd>只按 <code>PATH</code> 查找外部可执行文件；它会忽略 builtin 和函数。</dd></dl>
```

printf 与参数观察器

SYNOPSIS

```
<pre><code>printf format [arguments ...]</code></pre>
```

bash -c 'command_string' name [argument ...]

printf 用显式格式显示字符串边界；bash -c 启动新的 Bash，把 name 作为 \$0，其余参数依次成为 \$1、\$2 ……。

```
printf 'arg=<%s>\n' "$@"
```

逐个显示参数，并给每个参数增加可见边界。

```
"$#"
```

读取位置参数数量。

```
"$@"
```

在双引号内保留每个位置参数的独立边界。

set / export -p / env

SYNOPSIS

```
<pre><code>set
```

```
export [-fn] [name[=value] ...] env [name=value ...] [command [argument ...]]
```

三个入口分别观察当前 Shell 状态、带导出属性的变量和进程环境，不能互相替代。

```
set
```

显示当前 Shell 的变量和函数，范围最宽。

```
export -p
```

显示具有导出属性的变量。

```
env
```

显示当前进程环境，或给单条命令构造临时环境。

变量赋值、临时环境与清理

SYNOPSIS

```
<pre><code>name=value
```

```
name=value command unset name export -n name
```

赋值改变当前 Shell 变量；命令前赋值只为该次命令提供环境；删除变量和取消导出属性是两种不同操作。

```
name=value
```

等号两侧不能随意加入空格。

```
name=value command
```

为该命令提供临时环境，不等于永久持久配置。

```
unset name
```

删除变量。

```
export -n name
```

保留当前 Shell 变量值，但取消后续子进程继承属性。

```
 ; / && / ||
```

SYNOPSIS

```
<pre><code>command1 ; command2
```

```
command1 && command2 command1 || command2
```

使用前一命令的退出状态决定后续命令是否执行；`;` 不判断状态，`&&` 成功继续，`||` 失败继续。

```
status=$?
```

在执行任何其他命令前立即保存最近状态。

左结合

`a && b || c` 在 `a` 失败或 `b` 失败时都可能执行 `c`，不能无条件当成严格 `if/else`。

短路

被跳过的命令没有运行，也不会产生新的退出状态。

`( ... ) / { ...; } / $(...)`

SYNOPSIS

```
<pre><code>( list )
```

```
{ list; } result=$(command)
```

三种形式分别用于隔离执行环境、在当前 Shell 中分组，以及捕获子 Shell 命令输出。

`( list )`

在子 Shell 环境执行，变量和目录修改通常不保留。

`{ list; }`

在当前 Shell 执行；右花括号前必须有分号或换行，左右花括号需要词边界。

`$(command)`

捕获标准输出并嵌入当前词；若结果应保持一个参数，通常使用 `"$(command)"`。

[知识专题] 从输入文本到最终参数列表

Shell 排错最稳定的切入点不是先猜“这个命令的选项是不是写错了”，而是先重建最终参数列表。命令行中的空格、引号、美元符号和星号都可能影响参数数量或内容；只有知道程序实际收到什么，才能继续判断目标命令如何解释这些参数。

① [知识点] 命令名、选项和普通参数在 Shell 层都是词

下面的命令：

```
ls -l /etc/hosts
```

经过 Shell 处理后，可以近似理解为：

```
argv[0] = ls
argv[1] = -l
argv[2] = /etc/hosts
```

第一个词用于选择要执行的命令，后续词成为参数。`-l` 被称为选项，是因为 `ls` 按自身接口把它解释成选项；对 Bash 来说，它首先只是一个普通参数。Shell 不会自动理解每个程序的选项语义。

这一区分很重要：

- Bash 决定词边界、展开和命令组合；
- 目标程序决定参数是什么意思；
- 某个参数以 `-` 开头，不代表 Shell 会特殊处理它；
- 某个字符串看起来像路径，也不代表 Shell 会先检查它是否存在。

② [知识点] 未引用空白和控制操作符建立边界

通常情况下，空格和制表符分隔命令中的词：

```
printf '%s\n' alpha beta
```

这里 `printf` 收到格式参数和两个待格式化参数。若要让空格成为一个参数内部的数据，必须引用或转义：

```
printf '<%s>\n' 'alpha beta'
printf '<%s>\n' alpha\ beta
```

换行通常结束当前命令。反斜杠紧接换行时，二者一起被移除，用于把一条命令续写到下一行：

```
printf '%s\n' \
'a long argument'
```

`;`、`&&` 和 `||` 是控制操作符，它们分隔命令或命令列表，不会作为普通参数传给左侧命令。它们的条件语义将在退出状态专题中展开。

③ [知识点] 名称解析决定实际执行对象

同一个名称可能对应：

- `alias`；
- Shell 函数；
- Bash builtin；
- `PATH` 中找到的外部可执行文件。

使用：

```
type printf
type -a printf
type export
type bash
```

`type` 回答“当前 Bash 会把这个名称解析成什么”；`type -a` 尽可能显示所有可用入口。它比只查找文件路径更适合解释 Shell 语义，因为 builtin 和函数可能根本没有一个独立的外部文件路径。

命令被找到并不代表任务已经完成。`type` 只能证明解析入口，不能证明参数正确、权限足够或命令执行后的业务终态正确。

④ [操作] 使用参数观察器显示最终边界

`printf` 可以让不可见的参数边界变得可见：

```
printf 'arg=<%s>\n' one 'two words' ''
```

预期有三次格式应用，其中最后一个参数为空字符串。为了同时查看参数数量和每个位置，可使用一个只读的 Bash 参数观察器：

```
bash -c '  
printf "argc=%s\n" "$#"  
printf "arg=<%s>\n" "$@"  
' _ one 'two words' ''
```

这里 `_` 占据新 Bash 的 `$0`，其后的词依次成为 `$1`、`$2`……。观察器不解释目标程序的选项，只负责回答：

```
最终有多少个参数？  
每个参数的精确文本是什么？  
是否存在空参数？  
一个变量是否意外变成多个参数？
```

⑤ [知识点] 源文本、Shell 词和最终参数是三个不同层次

不要把下面三种状态混为一谈：

1. 源文本：你在终端输入的字符，包括引号和反斜杠；
2. Shell 词：解析时识别出的词，仍可能包含待展开表达式；
3. 最终参数：展开、拆分、路径名匹配和引用移除之后交给命令的字符串。

引号通常不会作为字符出现在最终参数中。它们主要用于控制解析和展开：

```
printf '<%s>\n' "hello world"
```

最终参数是 `hello world`，不是包含双引号字符的 `"hello world"`。若目标参数本身必须包含引号字符，就要把这些引号作为数据另行引用或转义。

先用 ``type`` 确认执行入口。
再用参数观察器确认 `argc` 和每个 `arg`。
Shell 决定参数边界；目标程序决定参数语义。

[知识专题] 引用与转义：决定哪些字符还是语法

引用不是最后给文本“加保护壳”，而是在 Shell 识别和展开命令时持续影响字符的含义。最顺的判断方法是先问目标：需要完全保持字面文本，还是允许变量和命令替换，但仍必须保持一个参数？

① [知识点] 反斜杠保护紧随其后的一个字符

未引用的反斜杠让下一个字符失去特殊含义：

```
printf '%s\n' Your\ cost
printf '%s\n' \$HOME
printf '%s\n' \*
```

分别产生：

- 一个包含空格的参数 `Your cost`；
- 字面 `$HOME`；
- 字面 `*`。

反斜杠只保护紧随其后的字符，不会自动保护后面的整段文本。反斜杠换行是特殊情况：它们组成续行标记，二者都不会进入最终参数。

② [知识点] 单引号保留内部所有字符的字面值

单引号中的 `$`、`*`、反斜杠和空格都作为普通字符：

```
printf '<%s>\n' '$HOME *.txt \n'
```

单引号内部不能直接嵌入单引号。需要字面单引号时，必须结束当前引用、提供一个被保护的单引号，再开始新的引用。例如：

```
printf '%s\n' 'don'\''t'
```

这仍然是一个 Shell 词，由三个相邻片段拼接而成：`'don'`、`\'` 和 `'t'`。

适合使用单引号的典型目标：

- 显示字面的 `$HOME` ；
- 保存含大量 Shell 特殊字符的固定模板；
- 防止变量、命令替换和 glob 发生。

③ [知识点] 双引号允许有限展开并保持一个词

双引号中仍会发生：

- 参数展开，例如 `"$HOME"` ；
- 命令替换，例如 `"$(date +%F)"` ；
- 某些反斜杠处理。

双引号会抑制普通的词拆分和路径名展开，因此变量值即使包含空格或星号，通常也会作为一个参数：

```
value='one two *'  
printf '<%s>\n' "$value"
```

双引号不是“所有特殊字符都失效”。它的核心用途是：允许必要的替换，同时保留结果的参数边界。

④ [知识点] 双引号中的反斜杠不是通用转义

在双引号内部，反斜杠只在有限字符前保留转义作用，常见的是 `$`、反引号、双引号、反斜杠和换行。其他字符前的反斜杠可能被保留为数据。

因此，不要把下面两种写法机械地理解成完全等价：

```
printf '%s\n' "$HOME"  
printf '%s\n' "\q"
```

第一条用于保护美元符号；第二条中的反斜杠通常仍会作为参数内容的一部分。实际写命令时，应让引用结构表达目标，不要堆叠无意义反斜杠。

⑤ [知识点] 相邻片段会拼接为同一个 Shell 词

Shell 允许一个词由多种引用上下文拼接：

```
prefix='daily'  
printf '<%s>\n' "$prefix"-report\ final
```

最终只有一个参数：

```
daily-report final
```

这个能力用于精确控制一个参数中哪些部分展开、哪些部分保持字面。例如：

```
printf '%s\n' 'literal-$HOME-'"$USER"
```

前半段保持字面，后半段允许变量展开，但它们仍共同组成一个参数。

⑥ [操作] 按目标选择引用方式

目标	推荐形式	原因
完全保持字面文本	'text'	阻止所有普通 Shell 展开
允许变量或命令替换，并保持一个参数	"\$value"、"\${command}"	抑制词拆分和 glob
只保护下一个字符	\\$、\ 	局部转义
变量目录含空格，但后面的模式仍需 glob	"\$dir"/*.txt	只引用变量部分
参数本身需要包含引号字符	用另一种引用或局部转义	语法引号不会自动进入参数

⑦ [边界] 引用解决的是 Shell 层问题，不自动验证数据语义

"\$path" 能保证路径值作为一个参数，却不能证明：

- 路径存在；
- 当前用户有权限；
- 目标程序把它解释为路径；
- 对该路径执行操作是安全的。

引用是参数完整性的必要工具，不是完整的业务验证。

Cheatsheet

完全字面化 → 单引号。
允许展开但保持一个参数 → 双引号。
只保护下一个字符 → 反斜杠。
语法引号通常不会进入最终参数。

[知识专题] 展开流水线：一个词为什么会改变内容或数量

Shell 展开最容易被错误地学成一串互不相关的符号。更稳定的模型是跟踪“一个词在每一步会发生什么”：它可能被替换成新的文本，未引用结果可能被拆成多个词，随后每个词中的 glob 模式又可能变成多个路径。

① [知识点] 引用上下文从解析阶段就影响展开

可以用下面的简化流水线理解本章范围：

识别词与引用上下文

- 参数展开和命令替换
- 对未引用展开结果进行词拆分
- 路径名展开
- 移除语法引用字符
- 形成最终参数

完整 Bash 还包括大括号、波浪号和算术展开等阶段。本章只轻量说明前两者，不扩展为完整脚本语言课程。

不要误解为“先删掉所有引号，再做展开”。引号在前面的解析和展开阶段已经决定哪些字符具有特殊含义；最后的引用移除只是把作为语法使用的引号和反斜杠从最终参数中去掉。

② [知识点] 参数展开读取变量值

最基本的参数展开形式：

```
name='servera'
printf '%s\n' "$name"
printf '%s\n' "${name}01"
```

花括号用于明确变量名边界。若写成：

```
printf '%s\n' "$name01"
```

Bash 会尝试读取名为 `name01` 的变量，而不是 `name` 后接字面 `01`。

本章只要求掌握 `$name` 和 `${name}` 的基本形式。`${parameter:-word}`、字符串截取、数组和间接展开等高级家族留给 Bash 脚本专题。

③ [知识点] 未设置、已设置为空和显式空参数不是同一状态

考虑：

```
unset value
empty=''
```

下面几种结果需要区分：

```
bash -c 'printf "argc=%s\n" "$#" _ $value'
bash -c 'printf "argc=%s\n" "$#" _ "$value"'
bash -c 'printf "argc=%s\n" "$#" _ "$empty"'
bash -c 'printf "argc=%s\n" "$#" _ ''
```

未引用的空展开可能不产生参数；双引号中的空展开或显式 `''` 会保留一个空参数。这个边界在命令要求一个“存在但为空”的字段时非常重要。

④ [知识点] 命令替换把命令标准输出嵌入当前词

推荐形式：

```
today=$(date +%F)
printf '%s\n' "$today"
printf '%s\n' "report-$(date +%F)"
```

`$(...)` 比反引号更易读、易嵌套。命令替换会移除替换结果末尾连续的换行；中间换行仍可能保留。若命令替换未被双引号包围，其结果还可能进行词拆分和路径名展开。

命令替换捕获的是文本输出，不是退出状态。退出状态仍由命令或赋值命令按 Bash 规则提供，并可通过 `$?` 读取。

⑤ [知识点] 命令替换在子 Shell 环境中执行

下面的修改不会按直觉保留到父 Shell：

```
place=$(cd /tmp; pwd)
pwd
```

`cd /tmp` 发生在命令替换创建的子 Shell 环境中；父 Shell 当前目录不因此改变。`place` 获得的是命令输出。

这一区分解释了两种不同目标：

```
需要捕获文本 → `$(...)`
需要修改当前 Shell 状态 → 在当前 Shell 直接执行或使用 `{ ...; }`
```

⑥ [知识点] 词拆分主要作用于未引用的展开结果

设：

```
value='one two'
```

比较：

```
bash -c 'printf "argc=%s\n" "$#"; printf "<%s>\n" "$@"' _ $value
bash -c 'printf "argc=%s\n" "$#"; printf "<%s>\n" "$@"' _ "$value"
```

第一条通常得到两个参数，第二条得到一个参数。源文本中的空白在词法分析阶段建立边界；变量展开结果中的空白，则可能在后续词拆分阶段依据 `IFS` 产生多个字段。

因此“变量含空格时要加双引号”只是表象，真正机制是：双引号抑制了展开结果的词拆分。

⑦ [边界] IFS 会改变未引用展开结果的拆分规则

IFS 是 Bash 用于字段拆分的变量。管理员环境中它通常保留默认空白字符集合。若它被异常修改，未引用变量可能出现难以理解的参数边界。

诊断时可以查看：

```
printf 'IFS=<%q>\n' "$IFS"
```

本章不教学如何用自定义 IFS 编写解析脚本；只要求知道它是参数数量异常的一项状态证据。不要为了“修复一个命令”在共享 Shell 中随意重设 IFS，应先确认来源和影响范围。

⑧ [知识点] 路径名展开把模式变为文件名列表

常见模式：

模式	含义
*	零个或多个字符
?	任意一个字符
[abc]	集合中的一个字符
[!abc]	不在集合中的一个字符
[[:digit:]]	当前 locale 中的数字字符类

例如：

```
printf '<%s>\n' /etc/*.conf
```

Bash 在执行 printf 前把模式替换成匹配路径列表。printf 自身并不会读取目录并解释星号。

⑨ [知识点] 默认 glob 有三个容易误判的边界

- 通常不匹配名称开头的点：* 不会自动包含 .hidden；
- 无匹配时通常保留原模式：程序可能收到字面 *.txt；
- 匹配顺序受 locale 等状态影响：不要把显示顺序当成跨环境不变的业务事实。

Bash 选项如 nullglob、failglob 和 dotglob 可以改变这些行为。RHEL 9 实机上的默认选项应通过真实 Shell 查询，应在目标 Shell 中查询，本章不预设其状态。

⑩ [操作] 只让需要的部分参与 glob

假设目录变量可能包含空格：


```
dir='/tmp/monthly reports'
```

正确的混合形式：

```
printf '<%s>\n' "$dir"/*.txt
```

解释：

- `"$dir"` 保持目录值为一个片段；
- `/` 和 `*.txt` 与它拼成同一个 Shell 词；
- 星号未引用，因此整个路径模式仍会参与路径名展开。

两个典型错误：

```
printf '<%s>\n' "$dir/*.txt"    # 星号被引用，成为字面字符
printf '<%s>\n' $dir/*.txt      # 目录变量先可能被词拆分
```

⑪ [知识点] 大括号展开不是文件匹配

```
printf '%s\n' file{1,2,3}.txt
printf '%s\n' node{01..03}
```

大括号展开按语法生成字符串，不检查这些文件或主机是否存在。它发生得较早，结果还可能继续经历后续展开。

本章只要求能区分：

```
`{a,b}` → 生成候选字符串
`[ab]` → 路径名模式中的一个字符集合
```

⑫ [知识点] 波浪号展开有自己的引用边界

未引用的 `~` 位于适当位置时可展开为主目录：

```
printf '%s\n' ~
printf '%s\n' ~/logs
```

引用后的波浪号通常保持字面：

```
printf '%s\n' "~/logs"
```

需要在变量或配置中稳定表示当前用户主目录时，`"$HOME/logs"` 往往比依赖引号外的 `~` 更清晰。

变量值要保持一个参数 → ``"$value"``。
目录变量含空格但仍需 glob → ``"$dir"/*.txt``。
无匹配模式可能作为字面参数传入，必须观察 `argv`。
``{a,b}`` 生成字符串；``[ab]`` 参与路径名匹配。

[操作专题] Shell 变量、导出属性与子进程环境

变量问题必须同时区分三个层次：当前 Shell 中是否有值、变量是否具有导出属性、真正启动的子进程是否收到了该环境项。只在父 Shell 中 `echo` 出一个值，不能证明外部程序也能看到它。

① [操作] 创建和修改当前 Shell 变量

基本形式：

```
name=value
count=40
path='/tmp/report data'
```

= 两侧不能出现用作分隔的空格：

```
name = value
```

这不会被解析成赋值，而会尝试执行名为 `name` 的命令并传入参数。变量名通常由字母、数字和下划线组成，不能以数字开头。变量名区分大小写。

赋值只改变当前 Shell 状态。两个独立终端各自运行不同 Shell，它们不会自动共享后续的普通变量修改。

② [查询] 使用 `set` 查看当前 Shell 状态

```
set
```

`set` 输出当前 Shell 的变量和函数，内容通常很多。已知变量名时，应优先精确显示：

```
printf 'name=<%s>\n' "$name"
```

需要判断变量是否存在而不是只看值时，完整脚本会使用更精细的参数展开或 `declare -p`；这些属于脚本语言扩展，本章只建立“值为空不等于一定未设置”的边界。

③ [操作] 使用 `export` 添加导出属性

```
mode=production
export mode
```

或一次完成：

```
export mode=production
```

`export` 不是把变量“发布到整个操作系统”，而是给当前 Shell 中的名称添加导出属性。此后由该 Shell 启动的子进程会在自己的初始环境中收到对应名称和值。

查看导出声明：

```
export -p
```

④ [查询] `set`、`export -p` 和 `env` 回答不同问题

命令	主要观察对象
<code>set</code>	当前 Bash 的变量和函数
<code>export -p</code>	当前 Bash 中带导出属性的名称
<code>env</code>	执行 <code>env</code> 这个进程实际收到的环境

诊断环境继承时，这三项可以形成证据链：

```
当前 Shell 是否有值
→ 是否具有导出属性
→ 子进程环境是否实际包含
```

⑤ [验证] 用新 Bash 检查真实继承

```
scope=private
bash -c 'printf "scope=<%s>\n" "$scope"'

export scope
bash -c 'printf "scope=<%s>\n" "$scope"'
```

第一条新 Bash 不应依赖未导出的普通变量；第二条在 `export` 后应能够读取。该方法验证的是新 Bash 进程的环境，仍不代表所有远程会话或将来登录都自动拥有这个变量。

⑥ [知识点] 子进程不能反向修改父 Shell

```
value=parent
bash -c 'value=child; printf "%s\n" "$value"'
printf '%s\n' "$value"
```

子进程可以修改自己的变量和环境，但父 Shell 的 `value` 仍保持原值。环境继承是父到子的初始复制，不是双向共享存储。

⑦ [操作] 给单条命令提供临时环境

```
MODE=test bash -c 'printf "%s\n" "$MODE"'
env MODE=test bash -c 'printf "%s\n" "$MODE"'
```

前置赋值针对这条简单命令的执行环境。它适合临时改变一个程序的 locale、模式或其他环境输入，而不必先永久导出到后续所有子进程。

验证边界：命令内部看到 `MODE=test`，不代表当前父 Shell 在命令完成后保留同样变量状态。

⑧ [操作] 删除变量与取消导出属性

```
unset mode
export -n mode
```

- `unset mode` 删除变量；
- `export -n mode` 取消导出属性，但变量值仍可留在当前 Shell。

操作后分别用 `printf`、`export -p` 和新的 `bash -c` 验证，不要只运行其中一条命令就假定全部状态都符合预期。

⑨ [边界] 当前变量不等于持久配置

在交互 Shell 中执行：

```
export EDITOR=vim
```

只改变当前 Shell 及其后续子进程。新的登录或新的独立终端是否得到该值，取决于 Bash 启动模式和启动文件配置。

启动文件完整加载顺序属于后续 Shell 环境专题。本章只建立接口：

```
当前 Shell 状态
≠ 新会话初始状态
≠ 系统范围持久配置
```

⑩ [安全边界] 不要把秘密值直接暴露在不必要的环境中

环境变量适合许多普通配置，但并不是无条件安全的秘密存储。进程环境可能被同一用户的诊断工具、转储或日志间接暴露。考试任务若没有要求，不要把口令、令牌等敏感值随意导出到广泛的子进程树。

Cheatsheet

```
`set` 看当前 Shell。  
`export -p` 看导出属性。  
`env` 或 `bash -c` 看子进程实际环境。  
当前 export 不等于新登录持久配置。
```

[操作专题] 从目标参数反向构造安全命令

复杂命令不应从“拼一整行字符串”开始，而应先写出目标参数表。每个参数是一个整体还是允许展开成多个路径，决定了变量、命令替换和 glob 应处于什么引用上下文。

① [操作] 先写目标 argv

假设目标程序应收到：

```
arg 1 = --label  
arg 2 = nightly report  
arg 3 = /tmp/data set/a.txt  
arg 4 = /tmp/data set/b.txt
```

再设计 Shell 文本：

```
label='nightly report'  
dir='/tmp/data set'  
command --label "$label" "$dir"/*.txt
```

这里 `--label`、`"$label"` 和每个匹配路径分别形成独立参数。

② [操作] 默认引用变量展开

变量值来自路径、用户名、日期标签或其他动态数据时，默认使用：

```
"$value"
```

只有明确希望一个变量值进一步被拆成多个参数或被当作 glob 模式时，才考虑省略引号；这类设计应有清晰证据和边界，不应依赖“当前值碰巧没有空格”。

③ [操作] 命令替换期望一个参数时放入双引号

```
stamp="$(date +%F)"
printf '<%s>\n' "report-$stamp"
```

或直接：

```
printf '<%s>\n' "report-$(date +%F)"
```

未引用命令替换可能因输出中的空白或 glob 字符改变参数数量。即使当前命令输出看起来简单，也应让引用表达长期意图。

④ [操作] 只对模式部分保留 glob 能力

```
"$dir"/*.conf
```

不要把整个路径模式放在双引号中，也不要让可能含空格的目录变量保持未引用。

执行修改性命令前，先将目标命令替换成参数观察器：

```
bash -c '
printf "argc=%s\n" "$#"
printf "arg=<%s>\n" "$@"
' _ "$dir"/*.conf
```

确认参数数量和每个路径后，再进入真正操作。

⑤ [边界] 不使用 `eval` 修补引用问题

把整条命令放入字符串，再交给 `eval` 会触发第二次 Shell 解析。变量值中的空格、引号、分号或命令替换语法可能在第二次解析时获得新的控制能力。

本章的默认策略是：

```
使用正常参数边界
→ 直接调用目标命令
→ 不把命令和数据混成一条待再次解析的字符串
```

⑥ [验证] 命令执行前后分别验证

执行前：

- `type` 确认入口；
- 参数观察器确认 `argv`；
- 检查 glob 是否匹配到预期集合；

- 检查变量和环境来源。

执行后：

- 立即保存退出状态；
- 使用目标对象的查询接口验证终态；
- 不把“参数看起来正确”扩大为“业务一定完成”。

Cheatsheet

先写目标参数表，再写 Shell 命令。
变量默认双引号。
只让确实需要的 glob 保持未引用。
不要用 `eval` 代替正确的参数设计。

[知识专题] 退出状态与 `;`、`&&`、`||`

Shell 不只传递字符串，也传递命令完成后的状态。命令组合的关键不是“上一条有没有输出”，而是上一条实际返回的退出状态。状态是易失的：一旦执行下一条命令，`$?` 就会被新状态覆盖。

① [知识点] `$?` 保存最近命令或命令列表的状态

```
some_command
status=$?
printf 'status=%s\n' "$status"
```

正确顺序是先保存，再执行用于显示或验证的其他命令。错误顺序：

```
some_command
printf 'finished\n'
printf 'status=%s\n' "$?"
```

最后打印的是第一个 `printf` 的状态，不再是 `some_command` 的状态。

② [知识点] `0` 和非零只按命令自身约定解释

Unix 命令通常使用：

<code>0</code>	按命令定义成功
非零	未成功或出现特定结果

但“成功”的范围由命令决定。例如，命令成功读取配置文件，不代表远端服务可用；创建文件的命令状态为 `0`，也不代表文件内容、权限和持久性都符合完整任务要求。

某些查询命令会用不同非零值区分“未找到”“发生错误”等情况。需要精确解释时，应查该命令帮助或手册，而不是把所有非零值都压缩成同一个原因。

③ [操作] ; 表示无条件顺序执行

```
command1; command2
```

无论 `command1` 状态如何，只要 Shell 能继续解析，都会尝试执行 `command2`。它适合表达单纯顺序，不适合用来保护依赖前一步成功的操作。

命令列表的最终状态通常来自最后实际执行的命令，因此：

```
false; true
```

整个列表的最终状态是 `true` 的状态，而不是前面的失败状态。

④ [操作] && 只在左侧成功时执行右侧

```
command1 && command2
```

若 `command1` 返回 0，执行 `command2`；否则跳过右侧。它适合表达“只有前置条件成功才继续”。

示例：

```
test -d /etc && printf '%s\n' 'directory exists'
```

这里只说明条件组合机制，`test` 的完整表达式语法不在本章展开。

⑤ [操作] || 只在左侧失败时执行右侧

```
command1 || command2
```

左侧非零时执行右侧。它常用于提供失败提示或最小替代动作：

```
test -r /etc/hosts || printf '%s\n' 'not readable'
```

不要把所有失败都立即连接到破坏性修复。`||` 只告诉你左侧非零，具体原因仍需证据。

⑥ [知识点] && 与 || 左结合

```
a && b || c
```


按下面的结构理解：

```
(a && b) || c
```

因此 `c` 在两种情况下可能执行：

1. `a` 失败；
2. `a` 成功，但 `b` 失败。

这不是严格等价于“如果 `a` 成功就执行 `b`，否则执行 `c`”。若 `b` 自身可能失败且不希望触发 `c`，应使用更明确的控制结构；完整 `if` 语法留给 Bash 脚本章节。

⑦ [知识点] 短路执行会影响哪些命令真正运行

```
false && printf '%s\n' never  
true  || printf '%s\n' never
```

右侧没有执行，因此不能假定它产生了输出、修改了状态或生成了自己的退出结果。一个列表的最终状态应根据最后实际执行的命令判断。

⑧ [验证] 分别验证状态和目标对象

推荐链路：

```
执行命令  
→ 立即保存 ` $? `  
→ 解释该状态能证明什么  
→ 查询目标对象当前状态  
→ 检查持久状态或功能状态（若任务要求）
```

本章的参数观察器可以证明 `argv`；退出状态可以证明命令的自报结果；它们都不能单独替代目标对象的最终验收。

Cheatsheet

```
` $? ` 只能读取最近状态，必须立即保存。  
`;` 无条件继续。  
`&&` 成功继续。  
`||` 失败继续。  
`a && b || c` 中，b 失败也会触发 c。
```

[知识专题] 命令分组与子 Shell 的必要边界

命令分组解决两个问题：把多条命令作为一个列表处理，以及控制这些命令是在当前 Shell 还是隔离的子 Shell 环境中执行。最重要的判断不是符号长什么样，而是变量和工作目录修改是否需要保留。

① [知识点] (list) 在子 Shell 环境执行

```
value=outside
(
    value=inside
    cd /tmp
    printf 'inside value=%s pwd=%s\n' "$value" "$PWD"
)
printf 'outside value=%s pwd=%s\n' "$value" "$PWD"
```

圆括号内能看到父 Shell 的初始状态，但其中的后续修改不会保留到父 Shell。它适合：

- 临时改变目录；
- 临时修改变量或 Shell 选项；
- 需要隔离一组命令副作用的场景。

② [知识点] { list; } 在当前 Shell 执行

```
value=outside
{
    value=inside
    cd /tmp
}
printf 'value=%s pwd=%s\n' "$value" "$PWD"
```

变量和目录修改会保留。语法边界必须准确：

- { 和 } 需要作为独立的保留字；
- 结束 } 前必须有分号或换行终止列表。

错误形式：

```
{ cd /tmp }
```

这里缺少在 } 前终止 cd /tmp 的分号或换行。

③ [知识点] 两种分组都产生退出状态

分组的退出状态来自其中最后实际执行的命令：

```
( true; false )
printf '%s\n' "$?"

{ false; true; }
printf '%s\n' "$?"
```

如果分组内部使用 `&&` 或 `||`，还必须先判断哪些命令实际执行。

④ [知识点] 命令替换也是子 Shell 边界，但目的不同

```
result=$(cd /tmp; pwd)
```

这里：

- 子 Shell 中执行 `cd` 和 `pwd`；
- 标准输出被捕获到 `result`；
- 父 Shell 工作目录保持不变。

普通圆括号分组主要用于隔离执行；命令替换主要用于把输出嵌入当前词。不要把两者都简化成“开了一个括号”。

⑤ [边界] 本章不提前展开重定向和管道

命令分组常与以下形式结合：

```
把整个分组的输出写入一个目标
把分组放在管道一侧
```

这些涉及标准输入输出、文件描述符和管道进程模型，归第 02 章。本章只建立：分组是一个命令列表，有自己的执行环境和退出状态。

⑥ [操作决策] 根据状态保留需求选择分组

```
需要隔离 `cd`、变量或选项修改 → `( ... )`
需要让修改留在当前 Shell      → `{ ...; }`
只需要获取命令输出             → `$(...)`
```

三者都要继续检查参数、输出和退出状态，不能仅凭符号完成验证。

圆括号：子 Shell，修改不保留。
花括号：当前 Shell，修改保留。
命令替换：子 Shell 中执行并捕获输出。

[诊断专题] 从异常结果反推解析阶段

Shell 错误诊断应沿着最有区分度的证据推进。不要反复给命令添加引号或反斜杠直到“看起来能用”；先确定实际命令入口、最终参数、环境继承和退出状态，再做最小修改。

① [诊断] 含空格变量被拆成多个参数

症状：目标命令报告参数过多，或路径被分成几段。

当前证据：

```
printf 'value=<%s>\n' "$value"
bash -c 'printf "arg=%s\n" "$#"; printf "arg=<%s>\n" "$@"' _ $value
```

假设：未引用参数展开经过词拆分。

下一条最有区分度的证据：对比 `argc`：

```
bash -c 'printf "arg=%s\n" "$#"; printf "arg=<%s>\n" "$@"' _ "$value"
```

最小修复：在需要一个参数的位置使用 `"$value"`。

再验证：参数数量、完整路径和目标命令终态分别检查。

② [诊断] 程序收到字面的 `*.txt`

症状：错误中直接出现星号，或参数观察器显示 `/path/*.txt`。

调查顺序：

```
模式是否被整体放入引号
→ 当前目录或目录前缀是否正确
→ 是否存在匹配项
→ `set -f` 是否关闭 glob
→ `nullglob` / `failglob` 是否改变行为
```

最小修复不一定是“去掉所有引号”。目录变量含空格时，应改为：

```
"$dir"/*.txt
```

③ [诊断] glob 匹配数量比预期多或少

检查：

- 模式中的 `*`、`?` 和字符类范围；
- 是否误用了大括号展开；
- 隐藏文件是否需要包含；
- 当前 locale 是否影响字符类和排序；
- 目录中真实对象集合。

用参数观察器先枚举实际匹配结果，不要直接把不确定模式交给删除、权限修改或批量移动命令。

④ [诊断] 父 Shell 有值，子命令看不到变量

证据链：

```
printf 'parent=<%s>\n' "$name"
export -p
bash -c 'printf "child=<%s>\n" "$name"'
```

假设：变量存在但未导出，或启动的并不是预期子进程。

最小修复：

```
export name
```

或只给目标命令临时环境：

```
name=value command
```

再验证：在真正目标进程或等价的新 Bash 中读取，而不是只在父 Shell 打印。

⑤ [诊断] 变量或工作目录在命令后恢复原值

症状：括号内 `cd` 或赋值显示成功，但后续 Shell 未保留。

调查：

```
是否使用 `( ... )`
→ 是否发生在 `$(...)` 内
→ 是否由另一个 `bash -c` 执行
→ 目标是否确实要求修改父 Shell
```

若目标是保留当前 Shell 状态，可改用直接执行或 `{ ...; }`。若目标只是隔离临时状态，则恢复原值正是正确行为，不应修复。

⑥ [诊断] `$?` 与预期命令不对应

症状：状态总是 0，或显示的是打印命令的结果。

最常见原因是状态被覆盖：

```
command
printf 'command finished\n'
status=$?
```

最小修复：

```
command
status=$?
printf 'status=%s\n' "$status"
```

再检查该状态能证明什么，并继续验证目标对象。

⑦ [诊断] `&&/||` 执行了意外分支

把表达式按左结合重新加括号：

```
a && b || c
↓
(a && b) || c
```

逐步记录状态：

```
a
sa=$?
b
sb=$?
```

实际排错时不要为了记录状态而改变原列表的执行条件；可以先使用等价、安全的测试命令复现，或把命令分别运行并记录。重点确认：是 `a` 失败，还是 `b` 在 `a` 成功后失败。

⑧ [诊断] 看起来执行了错误的命令

症状：参数正确，但行为与预期工具不同。

先运行：

```
type -a name
```

检查 alias、函数、builtin 和外部文件。必要时在受控诊断中使用明确路径或 `command builtin` 绕过函数/alias，但不要把绝对路径硬编码成所有场景的默认答案。

⑨ [诊断] 一条安全的通用调查链

症状

- ``type -a`` 确认入口
- 参数观察器确认 `argc/argv`
- 检查引用位置和展开类型
- 比较 ``set`` / ``export -p`` / 子进程环境
- 立即保存退出状态
- 判断当前 Shell 与子 Shell 边界
- 最小修复
- 重做参数、状态和目标终态验证

Cheatsheet

- 参数错 → 看 `argc/argv`。
- 变量错 → 比较当前值、导出属性和子进程环境。
- 状态错 → 立即保存 ``$?``。
- 修改未保留 → 检查是否进入子 Shell。

[经典任务] 精确构造并验证一个复杂参数列表

任务环境

当前登录用户在 Bash 中工作。实验环境已经准备好以下目录和文件：

```
/tmp/rhcsa01 data/  
├─ report 01.txt  
├─ report 02.txt  
└─ notes.md
```

当前 Shell 已有变量：

```
work='/tmp/rhcsa01 data'  
label='nightly report'  
scope='private'
```

`scope` 当前只是普通 Shell 变量，尚未导出。

提供以下只读参数观察器：

```
bash -c '  
printf "argc=%s\n" "$#"  
printf "arg=<%s>\n" "$@"  
' -
```

任务目标

在观察器后补充参数，使新 Bash 精确收到四个参数：

- 1. 字面文本 \$HOME ；
- 2. 一个参数 nightly report-YYYY-MM-DD ，日期由 date +%F 在执行时生成；
- 3. /tmp/rhcsa01 data/report 01.txt ；
- 4. /tmp/rhcsa01 data/report 02.txt 。

随后完成以下验证：

- 1. 证明 scope 未导出时，新 Bash 不能依赖它；
- 2. 导出 scope 后，证明新 Bash 能读取 private ；
- 3. 分别运行一个明确成功和一个明确失败的命令，并在执行其他命令前保存状态；
- 4. 使用 && 仅在成功后显示 SUCCESS-BRANCH ；
- 5. 使用 || 仅在失败后显示 FAILURE-BRANCH ；
- 6. 用变量赋值和 pwd 对比 (...) 与 { ...; } 的状态保留差异。

限制条件

- 不使用 eval ；
- 不把整条命令保存为字符串再执行；
- 不使用重定向或管道；
- 不修改或删除提供的文件；
- 不假设 glob 一定匹配，必须检查参数观察结果；
- 不把退出状态 0 扩大解释为完整业务终态。

验收矩阵

验收项	证据
参数总数	观察器显示 argc=4
字面美元符号	第一个参数为 \$HOME ，不是真实主目录
空格未拆分	第二个标签和每个文件路径保持完整
命令替换发生	第二个参数含实际日期，不含字面 \$()
glob 发生	后两个参数分别是两个 .txt 路径

验收项	证据
未混入非目标文件	<code>notes.md</code> 不在参数列表
未导出变量不可见	第一次子 Bash 不获得可依赖的 <code>scope</code>
导出后可见	第二次子 Bash 显示 <code>private</code>
状态保存及时	打印的是目标命令状态，而非后续 <code>printf</code> 状态
短路分支正确	成功只触发 <code>&&</code> ，失败只触发 <code> </code>
子 Shell 隔离	圆括号内变量和目录修改不保留
当前 Shell 修改	花括号内变量和目录修改按设计保留

[参考解答] 先重建参数，再验证环境和状态

① 调查当前对象和变量

先确认正在使用 Bash，并观察相关命令的解析入口：

```
type bash
type -a printf
type export
```

查看变量时使用明确边界：

```
printf 'work=<%s>\n' "$work"
printf 'label=<%s>\n' "$label"
printf 'scope=<%s>\n' "$scope"
```

这一步证明父 Shell 中变量当前有值，但还没有证明 `scope` 已导出，也没有证明 `glob` 能匹配目标文件。

② 构造四个目标参数

完整观察命令：

```
bash -c '
printf "argc=%s\n" "$#"
printf "arg=<%s>\n" "$@"
' _ \
'$HOME' \
"$label-$(date +%F)" \
"$work"/*.txt
```

逐项解释：

- `'$HOME'` 使用单引号，因此美元符号和变量名保持字面；
- `"$label-$(date +%F)"` 允许变量和命令替换，同时保持一个参数；
- `"$work"/*.txt` 只引用目录变量，保留 `*.txt` 的路径名展开；
- `glob` 应匹配两个 `.txt` 文件，因此最终总参数数为 4。

推荐验证结果形态：

```
argc=4
arg=<$HOME>
arg=<nightly report-YYYY-MM-DD>
arg=</tmp/rhcsa01 data/report 01.txt>
arg=</tmp/rhcsa01 data/report 02.txt>
```

这里的 `YYYY-MM-DD` 只表示执行时的实际日期格式，本章不预设一个固定日期。

③ 分析三个典型错误

错误一：

```
"$HOME"
```

这会得到真实主目录，而不是字面 `$HOME`。

错误二：

```
$label-$(date +%F)
```

标签含空格，未引用展开结果可能被拆成多个参数。

错误三：

```
"$work/*.txt"
```

星号在双引号中失去路径名展开能力，程序会收到字面模式。

还有一种危险修正：

```
$work/*.txt
```

目录变量自身未引用，会先因空格产生错误边界，不能作为正确方案。

④ 验证变量导出前后的差异

先确认导出声明中是否有 `scope`：

```
export -p
```

使用新 Bash 检查：

```
bash -c 'printf "child scope=<%s>\n" "$scope"'
```

然后导出并再次验证：

```
export scope
bash -c 'printf "child scope=<%s>\n" "$scope"'
```

也可以使用更明确的环境查询：

```
env
```

但 `env` 输出很长；针对具体变量时，新 Bash 的明确打印更容易审阅。验证结论应限制为：当前 Shell 后续启动的这个子 Bash 能否读取变量，不扩大为新的登录会话也会持久拥有该值。

⑤ 及时保存退出状态

使用 Bash builtin `true` 和 `false` 构造不产生破坏的明确状态：

```
true
ok_status=$?
printf 'true status=%s\n' "$ok_status"

false
fail_status=$?
printf 'false status=%s\n' "$fail_status"
```

关键是赋值紧跟目标命令。若先执行 `printf`，原状态会被覆盖。

⑥ 验证短路执行

```
true && printf '%s\n' 'SUCCESS-BRANCH'
false && printf '%s\n' 'SHOULD-NOT-RUN'

false || printf '%s\n' 'FAILURE-BRANCH'
true || printf '%s\n' 'SHOULD-NOT-RUN'
```

这些证据分别说明：

- `&&` 只在左侧状态为 0 时执行；
- `||` 只在左侧非零时执行；

- 被短路跳过的命令没有实际运行。

⑦ 对比圆括号和花括号

为避免永久改变到未知目录，先保存当前工作目录：

```
start_dir=$PWD
state=outside
```

圆括号：

```
(
    state=inside-subshell
    cd /tmp
    printf 'subshell state=<%s> pwd=<%s>\n' "$state" "$PWD"
)
printf 'after subshell state=<%s> pwd=<%s>\n' "$state" "$PWD"
```

父 Shell 中应仍为 `state=outside`，目录仍为 `start_dir`。

花括号：

```
{
    state=inside-current-shell
    cd /tmp
}
printf 'after group state=<%s> pwd=<%s>\n' "$state" "$PWD"
```

此时修改保留在当前 Shell。完成演示后恢复目录：

```
cd "$start_dir"
```

恢复动作也应检查退出状态和 `pwd`，不能假定路径永远可用。

⑧ 分层验收

参数层：argc=4，每个 arg 内容正确。
展开层：字面美元、变量、命令替换和 glob 各按预期发生。
环境层：export 前后子进程可见性不同。
状态层：成功为 0、失败非零，且保存未被覆盖。
组合层：&& 和 || 只执行正确分支。
Shell 层：圆括号隔离，花括号保留。
边界层：没有使用重定向、管道、eval 或破坏性文件操作。

[本章收束] 把每条命令还原成参数、环境和状态

完成本章后，面对一条含变量、空格、星号和命令替换的命令，不应只凭视觉判断。稳定的工作方法是：

1. 用 ``type -a`` 确认实际执行入口。
 2. 写出目标 `argv`：每个参数应是什么。
 3. 用单引号、双引号或反斜杠控制语法与数据边界。
 4. 跟踪参数展开、命令替换、词拆分和 `glob`。
 5. 用参数观察器核对 `argc` 和每个 `arg`。
 6. 用 ``set``、``export -p`` 和子 `Bash` 区分变量与环境。
 7. 命令后立即保存 ``$?``。
 8. 用 ``;``、``&&``、``||`` 表达明确的顺序或条件。
 9. 根据状态是否要保留，选择当前 `Shell` 或子 `Shell`。
 10. 再用目标对象的查询接口完成最终验收。

主要判断表

需求	首选形式
完全保持字面文本	<code>'text'</code>
变量展开后保持一个参数	<code>"\$value"</code>
捕获命令输出并保持一个参数	<code>"\$(command)"</code>
目录变量含空格且需要 <code>glob</code>	<code>"\$dir"/*.txt</code>
让子进程继承变量	<code>export name</code>
只给一条命令临时环境	<code>NAME=value command</code>
保存最近退出状态	<code>status=\$?</code> ，紧跟目标命令
无条件继续	<code>a; b</code>
成功后继续	<code>a && b</code>
失败后继续	<code>a b</code>
隔离变量和目录修改	<code>(...)</code>
在当前 <code>Shell</code> 保留修改	<code>{ ...; }</code>

向下一章交接

本章已经回答：命令如何形成参数、如何选择执行对象、如何返回状态。第 02 章《标准输入输出、重定向与管道》将在此基础上继续回答：

命令从哪里读取数据？
`stdout` 和 `stderr` 分别去哪里？
重定向操作符何时生效？
管道如何连接多个进程？

本章不把这些问题的答案提前压缩成几条符号，以避免把“参数解析”和“文件描述符连接”混为同一个机制。